GCP BIGQUERY - ASSIGNMENT

HCL ID: HCLTFP2170249

Name: DHARUN P

Roll Number: 23BTRCL014

College Name: Jain University

Batch: 2027

Creating Database – ECOMMERCE Database with Table Entries

Query:

```
CREATE SCHEMA IF NOT EXISTS `ecommerce`
OPTIONS(location = 'US');

CREATE TABLE IF NOT EXISTS `ecommerce.orders` (
   order_id STRING,
   customer_id STRING,
   order_date DATE,
   product_id STRING,
   product STRING,
   quantity INT64,
   unit_price FLOAT64,
   region STRING,
   status STRING
);

INSERT INTO `ecommerce.orders`
(order_id, customer_id, order_date, product_id,
 product, quantity, unit_price, region, status)

VALUES
('O001','C001','2026-08-05','P101','Laptop',1,55000,'South','Delivered'),
('O002','C001','2026-09-10','P102','Mouse',2,800,'South','Delivered'),
```

('O003','C001','2026-10-12','P101','Laptop',1,55000,'South','Delivered'),

('O004','C002','2026-08-15','P103','Keyboard',1,1500,'North','Delivered'),

('O005','C002','2026-09-16','P104','Monitor',1,12000,'North','Delivered'),

('O006','C002','2026-10-18','P103','Keyboard',2,1500,'North','Delivered'),

('O007','C003','2026-05-02','P101','Laptop',1,55000,'West','Delivered'),

('O008','C003','2026-06-07','P105','Headphones',1,2500,'West','Delivered'),

('O009','C004','2026-10-11','P104','Monitor',2,12000,'West','Delivered'),

('O010','C004','2026-10-22','P106','Tablet',1,20000,'West','Cancelled'),

('O011','C005','2026-04-01','P102','Mouse',1,800,'South','Delivered'),

('O012','C006','2026-10-15','P106','Tablet',1,20000,'South','Delivered'),

('O013','C007','2026-10-20','P105','Headphones',3,2500,'North','Delivered'),

('O014','C008','2026-09-08','P106','Tablet',1,20000,'West','Pending'),

('O003','C001','2026-10-12','P101','Laptop',1,55000,'South','Delivered');

Output:

Schema | Details | Preview | Insights | Lineage

Row	order_id	customer_id	order_date	product_id	product	quantity	unit_price	region
1	O011	C005	2026-04-01	P102	Mouse	1	800.0	South
2	O002	C001	2026-09-10	P102	Mouse	2	800.0	South
3	O006	C002	2026-10-18	P103	Keyboard	2	1500.0	North
4	O004	C002	2026-08-15	P103	Keyboard	1	1500.0	North
5	O013	C007	2026-10-20	P105	Headphones	3	2500.0	North
6	O008	C003	2026-06-07	P105	Headphones	1	2500.0	West
7	O009	C004	2026-10-11	P104	Monitor	2	12000.0	West
8	O005	C002	2026-09-16	P104	Monitor	1	12000.0	North
9	O010	C004	2026-10-22	P106	Tablet	1	20000.0	West
10	O014	C008	2026-09-08	P106	Tablet	1	20000.0	West
11	O012	C006	2026-10-15	P106	Tablet	1	20000.0	South
12	O003	C001	2026-10-12	P101	Laptop	1	55000.0	South
13	O007	C003	2026-05-02	P101	Laptop	1	55000.0	West
14	O003	C001	2026-10-12	P101	Laptop	1	55000.0	South
15	O001	C001	2026-08-05	P101	Laptop	1	55000.0	South

TASK 01: Remove the Duplicate Orders

Query:

```
SELECT DISTINCT *

FROM `ecommerce.orders`

ORDER BY order_id;
```

Output:

Query results

Chat | View all jobs | Save results | Open in

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	order_id	customer_id	order_date	product_id	product	quantity	unit_price	region	status
1	0001	C001	2026-08-05	P101	Laptop	1	55000.0	South	Delivered
2	0002	C001	2026-09-10	P102	Mouse	2	800.0	South	Delivered
3	0003	C001	2026-10-12	P101	Laptop	1	55000.0	South	Delivered
4	0004	C002	2026-08-15	P103	Keyboard	1	1500.0	North	Delivered
5	0005	C002	2026-09-16	P104	Monitor	1	12000.0	North	Delivered
6	0006	C002	2026-10-18	P103	Keyboard	2	1500.0	North	Delivered
7	0007	C003	2026-05-02	P101	Laptop	1	55000.0	West	Delivered
8	0008	C003	2026-06-07	P105	Headphones	1	2500.0	West	Delivered
9	0009	C004	2026-10-11	P104	Monitor	2	12000.0	West	Delivered
10	0010	C004	2026-10-22	P106	Tablet	1	20000.0	West	Cancelled
11	0011	C005	2026-04-01	P102	Mouse	1	800.0	South	Delivered
12	0012	C006	2026-10-15	P106	Tablet	1	20000.0	South	Delivered
13	0013	C007	2026-10-20	P105	Headphones	3	2500.0	North	Delivered
14	0014	C008	2026-09-08	P106	Tablet	1	20000.0	West	Pending

TASK 2: Find the Net Revenue

Query:

```
SELECT
   ROUND(SUM(quantity * unit_price), 2)
      AS net_revenue
FROM (
   SELECT DISTINCT *
   FROM `ecommerce.orders`
)
WHERE status = 'Delivered';
```

Output:

Query results

Job information | Results

Row	net_revenue
1	237900.0

TASK 03: Find the Top 3 Products in each Region

Query:

```
SELECT
   product,
   region
FROM (
   SELECT
      product,
      region,
      SUM(quantity * unit_price) AS total_sales
   FROM (
      SELECT DISTINCT *
      FROM `ecommerce.orders`
   )
   WHERE status = 'Delivered'
   GROUP BY product, region
)
QUALIFY ROW_NUMBER() OVER (
   PARTITION BY region
   ORDER BY total_sales DESC, product
) <= 3
ORDER BY region, total_sales DESC;
```

Output:

Query results

Job information | Results | Visualisation | JSON | E

Row	product	region
1	Monitor	North
2	Headphones	North
3	Keyboard	North
4	Laptop	South
5	Tablet	South
6	Mouse	South
7	Laptop	West
8	Monitor	West
9	Headphones	West

TASK 4: Identify Inactive Customers

Query:

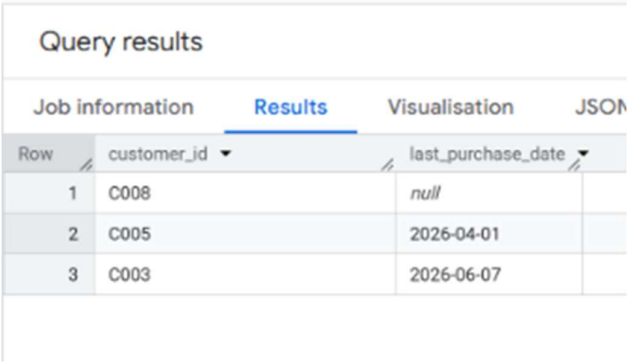
```
WITH customer_activity AS (
  SELECT
    customer_id,
    MAX(
      CASE
        WHEN status = 'Delivered'
        THEN order_date
      END
    ) AS last_purchase_date
  FROM (
    SELECT DISTINCT *
    FROM `ecommerce.orders`
  )
  GROUP BY customer_id
)
```

```
SELECT *
FROM customer_activity

WHERE last_purchase_date <
   DATE_SUB(CURRENT_DATE('Asia/Kolkata'), INTERVAL 90 DAY)
   OR last_purchase_date IS NULL

ORDER BY last_purchase_date;
```

Output:



Query results

Job information | Results | Visualisation | JSON

Row	customer_id	last_purchase_date
1	C008	*null*
2	C005	2026-04-01
3	C003	2026-06-07

TASK 5: Find the Customers who Purchase Continuously 3 Months

Query:

```
WITH monthly_orders AS (
   SELECT DISTINCT
      customer_id,
      DATE_TRUNC(order_date, MONTH) AS month
   FROM `ecommerce.orders`
   WHERE status = 'Delivered'
),
```

```
purchase_history AS (
    SELECT
        customer_id,
        month,
        LAG(month, 1) OVER (
            PARTITION BY customer_id
            ORDER BY month
        ) AS previous_month,
        LAG(month, 2) OVER (
            PARTITION BY customer_id
            ORDER BY month
        ) AS two_months_ago
    FROM monthly_orders
)

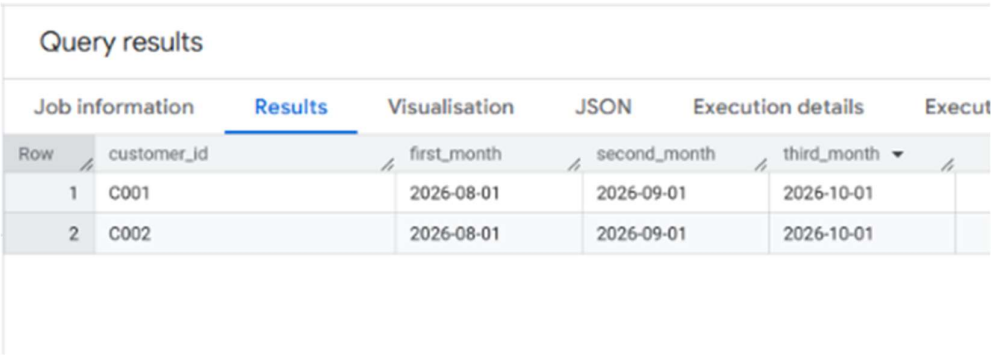
SELECT
    customer_id,
    two_months_ago AS first_month,
    previous_month AS second_month,
    month AS third_month
FROM purchase_history

WHERE previous_month =
    DATE_SUB(month, INTERVAL 1 MONTH)
```

```
AND two_months_ago =
   DATE_SUB(month, INTERVAL 2 MONTH)

ORDER BY customer_id;
```

Output:



Query results

Job information | Results | Visualisation | JSON | Execution details | Execut

Row	customer_id	first_month	second_month	third_month
1	C001	2026-08-01	2026-09-01	2026-10-01
2	C002	2026-08-01	2026-09-01	2026-10-01

TASK 6: Build a Customer Lifetime Value Report

Query:

```
SELECT
   customer_id,
   COUNT(DISTINCT order_id) AS total_orders,
   SUM(quantity) AS total_quantity,
   ROUND(SUM(quantity * unit_price), 2)
      AS customer_lifetime_value,
   ROUND(
      SAFE_DIVIDE(
         SUM(quantity * unit_price),
         COUNT(DISTINCT order_id)
      ), 2
   ) AS average_order_value,
```

```
    MIN(order_date) AS first_purchase,
    MAX(order_date) AS last_purchase

FROM (
    SELECT DISTINCT *
    FROM `ecommerce.orders`
)

WHERE status = 'Delivered'

GROUP BY customer_id

ORDER BY customer_lifetime_value DESC;
```

Output:

Query results

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	customer_id	total_orders	total_quantity	customer_lifetim...	average_order_va...	first_purchase	last_purchase
1	C001	3	4	111600.0	37200.0	2026-08-05	2026-10-12
2	C003	2	2	57500.0	28750.0	2026-05-02	2026-06-07
3	C004	1	2	24000.0	24000.0	2026-10-11	2026-10-11
4	C006	1	1	20000.0	20000.0	2026-10-15	2026-10-15
5	C002	3	4	16500.0	5500.0	2026-08-15	2026-10-18
6	C007	1	3	7500.0	7500.0	2026-10-20	2026-10-20
7	C005	1	1	800.0	800.0	2026-04-01	2026-04-01

TASK 7: Find the Monthly Sales

Query:

```
SELECT
    FORMAT_DATE('%Y-%m', order_date) AS sales_month,
    COUNT(DISTINCT order_id) AS total_orders,
    SUM(quantity) AS total_quantity,
```

```
    ROUND(SUM(quantity * unit_price), 2)
        AS monthly_sales

FROM (
    SELECT DISTINCT *
    FROM `ecommerce.orders`
)

WHERE status = 'Delivered'

GROUP BY sales_month

ORDER BY sales_month;
```

Output:

Query results

Job information | Results | Visualisation | JSON | Execution details | Execution

Row	sales_month	total_orders	total_quantity	monthly_sales
1	2026-04	1	1	800.0
2	2026-05	1	1	55000.0
3	2026-06	1	1	2500.0
4	2026-08	2	2	56500.0
5	2026-09	2	3	13600.0
6	2026-10	5	9	109500.0

TASK 8: Find the Unit Price Below 10000 in a Particular Month

Query:

```
SELECT DISTINCT
```

```
  order_id,
  customer_id,
  order_date,
  product,
  quantity,
  unit_price,
  region

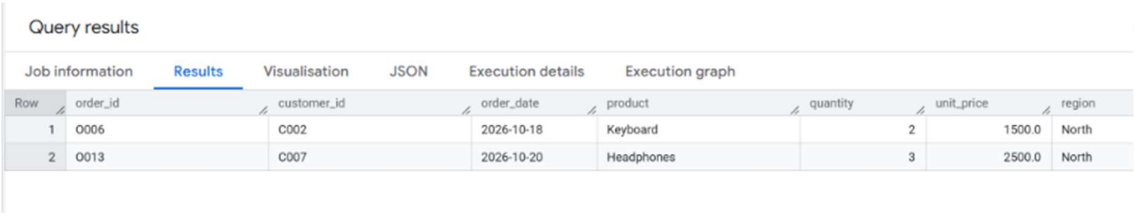
FROM `ecommerce.orders`

WHERE unit_price < 10000

AND EXTRACT(MONTH FROM order_date) = 10
AND EXTRACT(YEAR FROM order_date) = 2026

ORDER BY unit_price ASC;
```

Output:



Query results

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	order_id	customer_id	order_date	product	quantity	unit_price	region
1	0006	C002	2026-10-18	Keyboard	2	1500.0	North
2	0013	C007	2026-10-20	Headphones	3	2500.0	North

TASK 09: Find the Customer who is Purchased Particular Product Multiple Times in 6 Months

Query:

```
SELECT
  customer_id,
```

```
    product,
    COUNT(DISTINCT order_id) AS purchase_count,
    SUM(quantity) AS total_quantity,
    ROUND(SUM(quantity * unit_price), 2)
        AS total_spent

FROM (
    SELECT DISTINCT *
    FROM `ecommerce.orders`
)

WHERE product = 'Laptop'
AND status = 'Delivered'

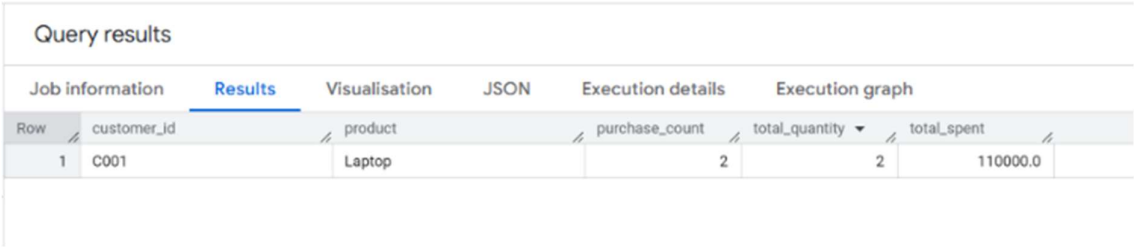
AND order_date BETWEEN
    DATE_SUB(
        CURRENT_DATE('Asia/Kolkata'),
        INTERVAL 6 MONTH
    )
    AND CURRENT_DATE('Asia/Kolkata')

GROUP BY customer_id, product

HAVING COUNT(DISTINCT order_id) > 1

ORDER BY purchase_count DESC;
```

Output:


Query results

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	customer_id	product	purchase_count	total_quantity	total_spent
1	C001	Laptop	2	2	110000.0

TASK 10: Find the Total Sales Report for each Product in a Particular Month

Query:

```
SELECT
   product_id,
   product,
   COUNT(DISTINCT order_id) AS total_orders,
   SUM(quantity) AS total_quantity,
   ROUND(SUM(quantity * unit_price), 2)
      AS total_sales

FROM (
   SELECT DISTINCT *
   FROM `ecommerce.orders`
)

WHERE status = 'Delivered'

AND EXTRACT(MONTH FROM order_date) = 10
```

```
AND EXTRACT(YEAR FROM order_date) = 2026

GROUP BY product_id, product

ORDER BY total_sales DESC;
```

Output:



Query results

Job information | Results | Visualisation | JSON | Execution details | Execution graph

Row	product_id	product	total_orders	total_quantity	total_sales
1	P101	Laptop	1	1	55000.0
2	P104	Monitor	1	2	24000.0
3	P106	Tablet	1	1	20000.0
4	P105	Headphones	1	3	7500.0
5	P103	Keyboard	1	2	3000.0